

# Python decorator in LangChain

A decorator is a function that modifies or enhances another function without changing its actual code.

```
from langchain.tools import tool

@tool
def retrieval....
```

For example:

```
from langchain.tools import tool

@tool
def calculator(a: int, b: int) -> int:
    return a + b
```

This is equivalent to:

```
from langchain.tools import tool

def calculator(a: int, b: int) -> int:
    return a + b

calculator = tool(calculator)
```

The `@tool` decorator converts a normal Python function into a LangChain Tool object that **an agent can understand and call**

LangChain extracts metadata such as:

- Tool name
  - Description (from the docstring)
  - Input schema
  - Return type
- and turn it into something that the agent can use
- For example:

```
@tool
def weather(city: str) -> str:
    """Get weather for a city."""
    return "30°C"
```

LangChain internally sees:

```
{
  "name": "weather",
  "description": "Get weather for a city.",
  "parameters": {
    "city": {
      "type": "string"
    }
  }
}
```

(In JSON format)

So the LLM then receives that tool definition and can decide:

User: What's the weather in Phnom Penh?

LLM:

→ Call tool "weather"

→ city = "Phnom Penh"

## Dataclass Decorator

It is from Python standard library

```
from dataclasses import dataclass

@dataclass
class Person: name: str age: int
```

It auto-generates boilerplate (repetitive, standard code that you have to write over and over again even though it does not really contain any unique logic) methods for you like

`__init__`, `__repr__`, and `__eq__` based on the class fields you define

**Without it:**

```
class Person:
    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age
```